

AI COURSE

COMPLETE EXAM PREP CHEAT SHEET

Punjab University College of Information Technology | All Lectures + Full Midterm Analysis

EXAM OVERVIEW — What Your Professor Tests

MIDTERM FORMAT

Part	Marks	Format	Details
Part 1	20	20 MCQs	Choose best option — must transfer answers to answer sheet or marks not counted
Part 2	50	5 Long-Answer Questions	Marks split per sub-part e.g. [1+2+4+3], [4+6], [2+4+4], [4+6], [2+6+2]

TOP TOPICS BY EXAM FREQUENCY

Rank	Topic	Frequency	Question Type
#1	Heuristic Properties (admissibility, monotonicity, informedness)	★★★★★	MCQ + long-answer proofs
#2	Alpha-Beta Pruning (α/β values, cut types, complexity)	★★★★★	MCQ + tree computation
#3	State Space Representation (4-tuple, start/goal, constraints)	★★★★	MCQ + draw graph
#4	BFS vs DFS (complexity, lists, properties)	★★★★	MCQ + analysis
#5	Best-First Search (algorithm, pseudocode)	★★★	MCQ + code analysis
#6	Minimax Algorithm (backing-up values, game trees)	★★★	MCQ + tree computation
#7	Genetic Algorithm (crossover, mutation, tournament selection)	★★★	MCQ + worked example
#8	PSSH + Data/Goal-Driven Search	★★	MCQ
#9	Tic-Tac-Toe state space + symmetry path counting	★★	MCQ — tricky math
#10	Hill Climbing + A* Search	★★	MCQ + True/False

TRICKY QUESTIONS — These caught students out on the midterm!

- Q1: "BFS always returns optimal solution" → FALSE. Only optimal when ALL move costs are equal.
- Q9: Worst-case complexity of α - β → $O(b^d)$, NOT $O(b^{(d/2)})$. That is best-case only.
- Q16: "Admissibility is stricter than monotonicity" → FALSE. Monotonicity is the stricter condition.
- Q18: "Larger tournament size = more diverse offspring" → FALSE. Larger = less diversity.
- Q8: α cut-off prunes below the current MIN node (answer B).
- Q12: State Space = CONCEPTUAL. Search Tree = CONCRETE (answer B).
- Q10: Admissibility does NOT imply monotonicity (answer D: none of given options).

■ TOPIC 1: Physical Symbol System Hypothesis (PSSH)

Definition

PSSH states: **"A physical symbol system has the NECESSARY and SUFFICIENT means for intelligent action."**

Proposed by **Newell and Simon** (1976 Turing Award). Means: a correctly programmed computer CAN be intelligent; and human thought is itself a symbol system.

■ *Analogy: Cooking. Symbols = ingredients. Expressions = recipe. Processes = cooking steps. Together they produce a meal. PSSH says intelligence works the same way.*

Examples of Physical Symbol Systems

System	Symbols	Expressions	Processes
Formal Logic	"and", "or", "not", "for all x"	True/false statements	Rules of logical deduction
Algebra	"+", "x", "y", numbers	Equations	Algebraic manipulation rules
Digital Computer	Zeros and ones in memory	Data structures	CPU operations
Chess	Pieces (king, queen...)	Board positions	Legal moves
Human Thought	Encoded in neurons	"Thoughts"	Mental operations
AI Program	Data	More complex data	Programs manipulating data

Four Attacks on PSSH (Nilsson's Paper)

Attack	Claim Against PSSH	Counter-Argument
1. Symbol Grounding	Computers shuffle meaningless symbols — intelligence needs real-world connection (embodiment)	PSSH already includes "designation": symbols CAN refer to real objects via perception/action
2. Non-Symbolic Processing	Perception uses analog signals, not discrete symbols	Non-symbolic processes SUPPLEMENT (not replace) symbol systems
3. Brain ≠ Computer	Brain is massively parallel, analog, fault-tolerant — not a serial computer	"Computation" has many description levels; brain scientists increasingly use computational concepts
4. Mindless Intelligence	Insects and evolution show intelligence without symbols	Mindless processes only produce mindless results; complex plans still require symbols

■ **EXAM CONNECTION:** Know (1) Newell & Simon coined PSSH, (2) necessity vs sufficiency, (3) the 4 attacks and counters, (4) examples of PSS (chess, algebra, logic, AI programs, human thought).

■ TOPIC 2: State Space Search

Definition

State Space = a graphical representation of ALL possible states of a problem, including every possible configuration and all solution states. Also called Solution Space or Problem Space.

■ *Analogy: A state space is like a map of every possible chess board position — every arrangement of pieces is a "state."*

The 4-Tuple Representation: State Space = [N, A, S, GD]

Symbol	Name	Meaning	8-Puzzle Example
N	Nodes / States	ALL possible configurations	Every arrangement of tiles 1-8 + blank
A	Arcs / Links	Legal moves between states	Slide tile up, down, left, or right
S	Start State(s)	Non-empty subset of N where search begins	The scrambled initial layout
GD	Goal State(s)	Desired end state(s)	Tiles in order: 1-2-3 / 4-5-6 / 7-8-blank

Data-Driven vs Goal-Driven Search

Property	Data-Driven (Forward Chaining)	Goal-Driven (Backward Chaining)
Direction	Start State → Goal State	Goal State → Start State
Analogy	Reading a book from page 1 to end	Solving a maze backwards from exit
How	Apply rules to facts; generate new facts until goal reached	Take goal; find rules that produce it; conditions become sub-goals
Best when	Start data is given; many possible goals	Goal is specific; many possible starts
Example	Given symptoms, diagnose condition	Given diagnosis, trace back to symptoms

■ Q4 Midterm: "Data-driven search = FORWARD CHAINING = searches from facts to goal(s)." Answer: C.

■ Q12 Midterm: State Space = CONCEPTUAL (abstract model). Search Tree = CONCRETE (actual tree built during search). Answer: B.

The 8-Puzzle Problem

3×3 grid with tiles 1-8 and one blank. Moves = sliding blank up/down/left/right. Reachable states from any start:

■ EXAM TRAP (Q6): Total arrangements = $9! = 362,880$. But only HALF are reachable from any start (due to puzzle parity).

■ **Correct answer: $9!/2 = 181,440$ reachable states. Answer: B.**

Common mistake: selecting $9!$ — that includes unreachable states too.

Tic-Tac-Toe State Space (Exam Favourite!)

3×3 board, X and O alternate. Counting paths from empty board to end of game:

Scenario	Formula	Value	Midterm Q
Without symmetry (total paths)	$9 \times 8 \times 7 \times \dots = 9!$	362,880	Q2
With symmetry of first placement	$8 \times 7!$	40,320	Q3

■ Q2 Answer C ($8!$): The board has 8 symmetric first placements, reducing total paths from $9!$.

■ Q3 Answer C ($8 \times 7!$): With symmetry, first move has 8 equivalent options, then $7!$ paths each.

These were the most missed questions on the midterm!

■ TOPIC 3: Uninformed (Blind) Search Algorithms

These algorithms search with NO knowledge about how close they are to the goal — like exploring a maze blindfolded.

1. Backtracking Search

Tries every path one at a time. When it hits a dead end, it backtracks to the most recent node with unexplored siblings.

■ *Analogy: Trying every key on a keychain one by one. If it fails, try the next.*

List	Full Name	What It Stores
SL	State List	Current path being explored. If goal found, SL IS the solution path.
NSL	New State List	Nodes awaiting evaluation (next candidates to explore).
DE	Dead Ends	States whose ALL descendants failed — never revisit.
CS	Current State	The single node currently being examined.

2. Breadth-First Search (BFS)

Explores level by level — all depth-1 nodes first, then depth-2, etc. Uses a FIFO queue for the OPEN list.

■ *Analogy: Ripples spreading from a stone dropped in water — the search spreads outward in equal waves.*

Property	BFS	Reason
Finds shortest path?	■ YES — when all move costs equal	Explores all shorter paths before any longer path
Memory usage	■ EXPONENTIAL — huge problem	Stores ALL nodes at current + all previous levels
Time complexity	$O(b^d)$	b = branching factor, d = depth of shallowest goal
Complete?	■ YES	Will always find a solution if one exists
Data structure	FIFO Queue	Ensures level-by-level exploration order

■■ EXAM TRAP (Q1): "BFS is guaranteed to return an optimal solution" → FALSE (B).

BFS only finds the optimal path when ALL action costs are equal. With varying costs → use Uniform Cost Search.

3. Depth-First Search (DFS)

Goes as deep as possible in one direction before backtracking. Uses a LIFO stack for the OPEN list.

■ *Analogy: Exploring a cave by always taking the first tunnel you see — going all the way to the end before trying another.*

Property	DFS	Reason
Finds shortest path?	■ NOT guaranteed	May find a long path before a shorter one
Memory usage	■ MUCH better than BFS	Only stores current path + backtrack points
Can get stuck?	■ YES — infinite loops	Can go down infinitely long paths
Best when?	Solution is known to be deep	Gets deep into search space quickly
Data structure	LIFO Stack	Most recently generated states explored first

■ TOPIC 3 (continued): Uninformed Search

4. Depth-First Iterative Deepening (DFID)

Best of both worlds! Performs DFS with an increasing depth limit. Gets memory efficiency of DFS AND the shortest-path guarantee of BFS.

■ *Analogy: Searching a building floor by floor — inspect floor 1 completely, then floors 1-2, then floors 1-3, etc.*

■ Q5: "DFID vs DFS for shallower trees?" → B: Equally well.

■ Q13: $d=3$, $b=2$. DFID worst-case nodes? Sum per iteration: depth-0=1, depth-1=3, depth-2=7, depth-3=15. Total: $1+3+7+15 = 26$. Answer: C.

■ TOPIC 4: Heuristic Search ★ HIGHEST EXAM WEIGHT ★

What is a Heuristic?

A **heuristic** is an educated guess — a rule of thumb that guides which path to explore next. Not guaranteed correct, but much faster than blind search.

■ *Analogy: Driving to a new city — you head roughly north-east. Not guaranteed perfect, but better than random.*

The Evaluation Function: $f(n) = g(n) + h(n)$

$$f(n) = g(n) + h(n)$$

Symbol	Name	Meaning	8-Puzzle Example
$f(n)$	Total evaluation	Overall score — lower = better	Total estimated moves to solve from n
$g(n)$	Actual cost	Real cost from START to node n	Number of moves made so far
$h(n)$	Heuristic	Estimated cost from n to GOAL	Tiles out of place, or Manhattan distance
$h^*(n)$	True cost	The ACTUAL optimal cost from n to GOAL	Real minimum moves needed to solve from n

■ Q15 Midterm: Uniform Cost Search = $f(n) = g(n) + h(n)$ where $h(n) = 0$. Answer: A.

Property 1: Admissibility

Definition: Heuristic $h(n)$ is admissible if it NEVER OVERESTIMATES the true cost to reach the goal.

Admissibility condition: $h(n) \leq h^*(n)$ for ALL states n

■ *Analogy: An optimistic GPS that always says "we are AT MOST X miles away." It might be even closer, but never falsely says we are farther.*

■ If h is admissible, A^* is GUARANTEED to find the optimal (shortest) path.

Example h_1 : tiles out of place in 8-puzzle. Each misplaced tile needs at least 1 move $\rightarrow h_1$ never overestimates $\rightarrow$ admissible.

Example h_2 : Manhattan distance (sum of moves each tile needs). Also admissible, and more informed than h_1 .

Property 2: Informedness

Definition: Heuristic h_2 is MORE INFORMED than h_1 if $h_1(n) < h_2(n)$ for all states n .

$h_1(n) < h_2(n)$ for all $n \rightarrow h_2$ is more informed than h_1

■ *Analogy: Two GPS apps. One says "10-30 minutes." The other says "exactly 14 minutes." The second is more informed (closer to the truth without overestimating).*

■ More informed heuristic = expands FEWER nodes = finds goal faster.

8-Puzzle: $h_1 = 4$ tiles out of place. $h_2 = \text{Manhattan distances} = 5$. Since $h_2 > h_1$: h_2 is more informed. Both still admissible!

TOPIC 4 (continued): Heuristic Properties

Property 3: Monotonicity (Consistency)

Definition: Heuristic h is monotonic if for any state n_i and its descendant n_j :

$$h(n_i) - h(n_j) \leq \text{cost}(n_i, n_j) \text{ AND } h(\text{goal}) = 0$$

■ *Analogy: A GPS updating "distance remaining" as you drive. Monotonic means the remaining distance can never INCREASE by more than the actual distance you drove. It decreases consistently.*

★ **KEY RELATIONSHIP: Monotonicity $\rightarrow$ Admissibility (if monotone, ALWAYS admissible).**

★ Admissibility does NOT guarantee Monotonicity (one-way relationship).

★ Monotonicity is STRICTER than admissibility.

PROOF (Midterm Q2b — 6 marks):

Given monotone condition: $h(n_i) - h(n_j) \leq \text{cost}(n_i, n_j)$

Apply along a full path $n = n_0, n_1, \dots, n_k = G$ and sum all inequalities:

$$h(n_0) - h(G) \leq \text{cost}(n_0, n_1) + \text{cost}(n_1, n_2) + \dots = h^*(n) \text{ (actual path cost)}$$

Since $h(G) = 0$: $h(n) \leq h^*(n) \leftarrow$ This IS admissibility! $\therefore$ Monotone $\rightarrow$ Admissible ✓

■ Q10: "Admissibility always implies ____" $\rightarrow$ D: None of given options. Admissibility does NOT imply monotonicity.

■ Q16: "Admissibility is stricter than monotonicity" $\rightarrow$ FALSE. MONOTONICITY is the stricter condition.

■ Q2 Part-2 (6 marks): Write out the full proof above step-by-step.

Heuristic Algorithms Compared

Algorithm	Uses	Optimal ?	Complete?	Key Feature
Best-First	$h(n)$ only — priority queue	■ No	■ No	Re-sorts queue by heuristic when new path added (Q11)
A* Search	$f(n)=g(n)+h(n)$, admissible h	■ Yes	■ Yes	Optimal if h admissible. The gold standard.
Hill Climbing	$h(n)$ only — greedy, no backtrack	■ No	■ No	Gets stuck in local optima. GA is a variant (Q19).
Uniform Cost	$g(n)$ only — $h(n)=0$	■ Yes	■ Yes	Special case of A* with $h(n)=0$ (Q15).

■ Q11: In best-first search "entire queue sorted each time new path added" $\rightarrow$ Answer B. ✓

■ Q14: "Hill climbing always guaranteed optimal?" $\rightarrow$ FALSE (B). Gets stuck in local optima.

■ Q19: "Genetic algorithm = variant of ____" $\rightarrow$ C: Hill Climbing search. ✓

■ TOPIC 5: Adversarial Search — Minimax & Alpha-Beta Pruning

What is Adversarial Search?

Search where TWO players alternate turns, each trying to WIN. Computer plays as **MAX** (maximising score). Opponent plays as **MIN** (minimising MAX's score). Used for chess, checkers, tic-tac-toe.

■ *Analogy: Arm wrestling — you pull as hard as possible (MAX), they resist as hard as possible (MIN). Both make rational choices.*

Game type: 2-player, zero-sum, perfect information, deterministic (no dice). Zero-sum = what one player gains, the other loses exactly.

Static Evaluation Function

Scores how good a board position is. Large positive = good for MAX. Large negative = good for MIN. Near 0 = neutral.

Tic-Tac-Toe: $f(n) = [\text{3-lengths open for P1}] - [\text{3-lengths open for P2}]$

A "3-length" = any row, column, or diagonal not yet blocked by the opponent.

The Minimax Algorithm

Step	Action
1	Create root node as MAX (your turn) with current board.
2	Expand nodes downward to a chosen depth limit (called "ply").
3	Apply static evaluation function to every LEAF node.
4	Back up values upward: MIN nodes take MINIMUM of children; MAX nodes take MAXIMUM of children.
5	Pick the move (arc) leading to the child whose backed-up value equals the root's value.

Worked Minimax Example (2-ply, from lecture):

Node	Type	Children Values	Backed-up Value	Reason
A	MIN	100, 3, -1	-1	$\min(100, 3, -1) = -1$
B	MIN	6, 5, 2	2	$\min(6, 5, 2) = 2$ (lecture shows B=5 — check your notes)
C	MIN	9	9	Only one child
S (root)	MAX	A=-1, B=5, C=2	5	$\max(-1, 5, 2) = 5 \rightarrow$ Move: S→B selected

Alpha-Beta Pruning — Smarter Minimax

Key guarantee: Alpha-Beta computes the EXACT SAME minimax value as Minimax, but skips branches that cannot change the final decision.

Value	At Node	Meaning	Behaviour
ALPHA (α)	MAX nodes	Best score MAX can guarantee so far	Monotonically NON-DECREASING (only goes up)
BETA (β)	MIN nodes	Best score MIN can guarantee so far	Monotonically NON-INCREASING (only goes down)

Initial values at root: $\alpha = -\infty$, $\beta = +\infty$

When to PRUNE:

Cut Type	Trigger Condition	What Gets Pruned
ALPHA cutoff (α -cut)	At MIN node n: $\beta(n) \leq \alpha$ of an ancestor MAX node	Remaining children of that MIN node (prune BELOW MIN)
BETA cutoff (β -cut)	At MAX node n: $\alpha(n) \geq \beta$ of an ancestor MIN node	Remaining children of that MAX node (prune BELOW MAX)

■ Q8: α cut-off prunes below the current MIN node (B). NOT the MAX node.

■ Q7: α - β always better than Minimax $\rightarrow$ TRUE (A). Q9: Worst-case α - $\beta \rightarrow$ A: $O(b^d)$.

Alpha-Beta Efficiency

Scenario	Nodes Examined	Implication
Worst case (no pruning at all)	$O(b^d)$ — same as Minimax	No improvement — moves ordered worst-first
Best case (perfect move ordering)	$O(b^{(d/2)})$	Search TWICE AS DEEP as Minimax with same resources!
Chess (Deep Blue empirical)	Avg branching ~6 instead of 35-40	Massive real-world improvement

Best case: $O(b^{(d/2)}) = O(\sqrt{b}^d)$

■ Q4 Part-2 (a): Alpha-Beta best case = $O(b^{(d/2)})$. Worst case = $O(b^d)$.

■ Q4 Part-2 (b): Compute (α, β) pairs — traverse LEFT to RIGHT, depth-first. Root starts $(-\infty, +\infty)$. Update α at MAX nodes (take max). Update β at MIN nodes (take min). Prune when $\alpha \geq \beta$.

From midterm solution: MIN selects node b because an alpha cut-off occurs at node a towards c.

■ TOPIC 6: Genetic Algorithm (GA)

What is a Genetic Algorithm?

An optimisation algorithm inspired by biological evolution. Maintains a POPULATION of candidate solutions, selects the best, combines them (crossover), and randomly tweaks them (mutation) to evolve better solutions.

Key exam fact: GA is a powerful variant of Hill Climbing (Q19 answer: C).

■ *Analogy: Breeding racing dogs — pick the fastest parents, breed them, occasionally introduce random traits to avoid all dogs becoming identical.*

GA Key Operations

Operation	What It Does	Purpose
Selection	Choose better individuals to be parents	Keep strong solutions in the gene pool
Crossover	Combine two parent solutions to create offspring	Share good features from both parents
Mutation	Randomly change part of a solution	Maintain DIVERSITY; prevent premature convergence
Fitness Fn.	Score how good each solution is	Determines who survives to next generation
Population	Collection of all current candidate solutions	Allows parallel exploration of solution space

Tournament Selection

Randomly pick k individuals (the "tournament"), select the best one as a parent. Repeat to get the second parent.

■ EXAM TRAP (Q18): "Larger tournament = more diverse offspring?" → FALSE.

Larger tournament = MORE selection pressure = only best survive = LESS diversity.

SMALLER tournament = more diversity (weaker individuals also get selected sometimes).

■ Q20: "Mutation ensures ___ in offspring" → B: Diversity. ✓

■ Q17: "TSP is NP-hard?" → TRUE (A). ✓

Order Crossover (OX) for TSP — Midterm Q5b

TSP requires visiting all cities exactly once. Standard crossover breaks this. Use Order Crossover:

Step	Action	Example with P1=(1,9,2,4,6,5,7,8,3), P2=(4,5,9,1,8,7,6,2,3)
1	Choose two cut points	Cut between positions 3&4 and 7&8
2	Copy P1 segment between cuts to child C1	C1 inherits positions 4-7: {4, 6, 5, 7}
3	Fill remaining positions from P2 order, skipping used cities	From P2 skip {4,6,5,7} → fill with 5,9,1,8,2,3
4	Result: valid tour, all cities appear exactly once	C1 is a valid TSP solution

■ CROSSOVER ISSUE: Standard crossover may REPEAT cities (invalid tour). Use Order Crossover (OX).

■ MUTATION ISSUE: Bit-flip eliminates cities (must visit ALL). Use SWAP mutation instead.

■ **SWAP mutation: swap two city positions in the tour — all cities still appear exactly once.**

■ COMPLETE MIDTERM MCQ ANALYSIS (Questions 1–10)

Q#	Question (Summary)	Correct Answer	Key Explanation
1	BFS guaranteed optimal if all move costs equal?	B: FALSE	BFS optimal ONLY with equal costs. For varying costs use Uniform Cost Search.
2	3×3 tic-tac-toe: # different paths start to end?	C: 8!	Board has 8 symmetric first placements reducing from 9! total.
3	With symmetry of first placement: # paths?	C: $8 \times 7!$	8 equivalent first moves due to board symmetry, then 7! paths each.
4	Which is true about data-driven search?	C: From facts to goal(s)	Data-driven = forward chaining. Starts from FACTS; generates new facts; reaches GOAL.
5	Shallower tree: DFID vs DFS?	B: Equally well	For shallow trees both are comparable. DFID advantage shows for deeper trees.
6	8-puzzle states accessible from any start?	B: $9!/2$	$9!/2 = 181,440$ reachable states. Only half of all 9! arrangements are reachable (parity).
7	α - β always better than minimax?	A: TRUE	α - β gives exact same result with fewer nodes. Always at least as efficient.
8	α cut-off means prune below current ____ node?	B: MIN node	α -cutoff triggers at MIN node when $\beta(\text{MIN}) \leq \alpha(\text{ancestor MAX})$. Prunes below that MIN.
9	Worst-case space complexity of α - β ?	A: $O(b^d)$	Worst case = no pruning = same as Minimax. $O(b^{(d/2)})$ is best case, not worst.
10	Admissibility always implies?	D: None of given options	Admissibility does NOT imply monotonicity. Reverse is true: monotone $\rightarrow$ admissible.

■ COMPLETE MIDTERM MCQ ANALYSIS (Questions 11–20)

Q#	Question (Summary)	Correct Answer	Key Explanation
11	In best-first search, which is true?	B: Queue sorted when new path added	Best-first re-orders the priority queue by heuristic value each time a path is added.
12	___ is conceptual; ___ is concrete.	B: State Space, Search Tree	State Space = abstract problem model. Search Tree = actual tree built during search.
13	Tree $d=3$, $b=2$: DFID nodes worst case?	C: 26	Sum: $1 + 3 + 7 + 15 = 26$. Each iteration re-expands to its depth limit.
14	Hill climbing always optimal (e.g. knapsack)?	B: FALSE	Hill climbing gets stuck in LOCAL OPTIMA. No backtracking means no global guarantee.
15	Uniform cost search heuristic form?	A: $f(n)=g(n)+h(n)$ where $h(n)=0$	Uniform cost only uses actual cost $g(n)$. Setting $h(n)=0$ gives this form.
16	Admissibility is stricter than monotonicity?	B: FALSE	MONOTONICITY is stricter. Monotone $\rightarrow$ admissible. Admissible does NOT $\rightarrow$ monotone.
17	TSP optimisation is NP-hard?	A: TRUE	TSP is a classic NP-complete/hard problem. No known polynomial-time solution.
18	Larger tournament size = more diverse offspring?	B: FALSE	Larger tournament = more selection pressure = only strongest survive = LESS diversity.
19	Genetic algorithm = variant of ___?	C: Hill Climbing	GA is a stochastic, population-based extension of hill climbing.
20	Mutation ensures ___ in offspring?	B: Diversity	Mutation prevents all solutions converging to same values — maintains genetic variety.

■ MIDTERM PART-2 LONG ANSWER ANALYSIS

Q1 [1+2+4+3 = 10 marks]: Farmer-Fox-Goose-Grain River Crossing

Problem: Farmer, fox, goose, grain must cross. Boat holds farmer + one item. Fox eats goose; goose eats grain if left unattended together.

State: where $a1=$ farmer, $a2=$ fox, $a3=$ goose, $a4=$ grain, each $\in \{L, R\}$

Part	Question	Answer
(a) 1 mark	Start state?	— all four on the left bank
(a) 1 mark	Goal state?	— all four on the right bank
(b) 2 marks	States to AVOID?	1) $a1 \in \{R\}$ and $(a2, a3) \in \{L\}$ — fox and goose alone on left 2) $a1 \in \{R\}$ and $(a3, a4) \in \{L\}$ — goose and grain alone on left 3) $a1 \in \{L\}$ and $(a2, a3) \in \{R\}$ — fox and goose alone on right 4) $a1 \in \{L\}$ and $(a3, a4) \in \{R\}$ — goose and grain alone on right
(d) 3 marks	Move sequence to goal	Step 1: Take goose across → Step 2: Return alone → Step 3: Take fox across → Step 4: Return with goose → (take goose back!) Step 5: Take grain across → Step 6: Return alone → Step 7: Take goose across → GOAL! ✓

Q2 [4+6 = 10 marks]: Heuristic Properties

(a) 4 marks — Differentiate Informedness vs Admissibility:

Property	Informedness	Admissibility
Compares	Two heuristics against each other	One heuristic against true cost $h^*(n)$
Condition	$h1(n) < h2(n) \rightarrow h2$ more informed	$h(n) \leq h^*(n)$ for all n
Achieves	More informed = fewer nodes expanded	Admissible = A^* finds optimal solution
Relation	Independent concept	Monotone $\rightarrow$ Admissible (not reverse)

(b) 6 marks — Prove Monotonicity implies Admissibility:

Given: $h(n_i) - h(n_j) \leq \text{cost}(n_i, n_j)$ for all n_i and descendant n_j [monotone condition]

Consider path $n = n_0, n_1, n_2, \dots, n_k = G$ from state n to goal G .

Apply monotone condition to each step and sum all inequalities:

$$h(n_0) - h(n_1) \leq \text{cost}(n_0, n_1)$$

$$h(n_1) - h(n_2) \leq \text{cost}(n_1, n_2)$$

...

$$h(n_{k-1}) - h(G) \leq \text{cost}(n_{k-1}, G)$$

Summing: $h(n_0) - h(G) \leq [\text{sum of all step costs}] = h^*(n)$ (actual optimal path cost)

Since $h(G) = 0$: $h(n) \leq h^*(n) \leftarrow$ Admissibility condition! $\therefore$ Monotone $\rightarrow$ Admissible ✓

■ MIDTERM PART-2 ANALYSIS (cont.)

Q3 [2+4+4 = 10 marks]: Best-First Search Pseudocode Analysis

Part	Question	Answer
(a) 2 marks	Does the pseudocode work?	NO. It removes the "leftmost state" from OPEN, but best-first should remove the state with the BEST (minimum) heuristic value.
(b) 4 marks	Fix the bug	Change: "remove the leftmost state from open" To: "remove the node nearest to the goal — the one with minimum $h(n)$ value (least underestimate of goal distance)"
(c) 4 marks	Modify for monotone heuristic	Remove the CLOSED LIST check entirely. With a consistent (monotone) heuristic, the first time we reach a node is already optimal — we never need to reconsider nodes on CLOSED.

Q4 [4+6 = 10 marks]: Alpha-Beta Pruning

Part	Question	Answer
(a) 4 marks	How does α - β improve Minimax? Give node counts.	Prunes branches that CANNOT affect the decision. When MAX already has a better option (α) than what MIN will allow (β), skip that branch. Best case: $O(b^{d/2})$ — when best moves are generated first. Worst case: $O(b^d)$ — no pruning; same as Minimax.
(b) 6 marks	Compute (α, β) pairs on given tree	Traverse left-to-right, depth-first. Root (MIN turn) starts with $(-\infty, +\infty)$. At MAX nodes: update $\alpha = \max(\alpha, \text{child value})$. At MIN nodes: update $\beta = \min(\beta, \text{child value})$. Prune when $\alpha \geq \beta$ at any level. From midterm: MIN selects node b; alpha cut-off at node a towards c.

Q5 [2+6+2 = 10 marks]: Genetic Algorithm for TSP

Part	Question	Answer
(a) 2 marks	Issues with bit-level representation for TSP?	CROSSOVER: Some cities may repeat in offspring (invalid tour). Fix: use Order Crossover (OX). MUTATION: Bit-flip eliminates cities from tour. Fix: use Swap Mutation (swap two positions).
(b) 6 marks	Apply order-crossover (P1, P2 given)	P1=(1,9,2,4,6,5,7,8,3), P2=(4,5,9,1,8,7,6,2,3) Cut between index 3&4 and 7&8. C1: inherit P1 segment {4,6,5,7} at positions 4-7. Fill rest from P2 order skipping used cities. C2: inherit P2 segment {1,8,7,6} at positions 4-7. Fill rest from P1 order skipping used cities.
(c) 2 marks	Tournament selection — what if size too large?	Tournament: pick k random individuals; select best as parent. If size too large: very high selection pressure → only strongest survive → LESS diverse offspring → premature convergence.

■ ALL KEY FORMULAS — Quick Reference

Formula	Name / Purpose	When to Use	On Midterm?
$f(n) = g(n) + h(n)$	Heuristic evaluation function	A* and best-first to rank nodes	■ Q15
$h(n) \leq h^*(n)$	Admissibility condition	Verify heuristic never overestimates	■ Q10, Q2-P2
$h_1(n) < h_2(n) \rightarrow h_2$ more informed	Informedness comparison	Compare two heuristics	■ Q2-P2
$h(n_i) - h(n_j) \leq \text{cost}(n_i, n_j)$	Monotonicity condition	Prove monotone $\rightarrow$ admissible	■ Q16, Q2-P2
$h(\text{goal}) = 0$	Monotonicity base condition	Always required for monotone heuristic	■ Q2-P2
$O(b^d)$	BFS / DFS / worst-case α - β complexity	Time/space complexity questions	■ Q9
$O(b^{d/2})$	Best-case α - β complexity	Best case — moves perfectly ordered	■ Q4-P2
$9!/2 = 181,440$	8-puzzle reachable states	How many states accessible	■ Q6
$f(n) = g(n) + 0 = g(n)$	Uniform Cost Search	Special case of A* with $h(n)=0$	■ Q15
$\alpha=-\infty, \beta=+\infty$ at root	Alpha-Beta initialisation	Always initialise root this way	■ Q4-P2
$9! = 362,880$	Tic-tac-toe without symmetry	Total game paths no symmetry	■ Q2
$8 \times 7!$	Tic-tac-toe with symmetry	After applying 8 board symmetries	■ Q3

■ KEY CONCEPT COMPARISONS

Concept A	Concept B	Key Difference	Relationship
BFS	DFS	BFS: level-by-level, optimal, huge memory. DFS: depth-first, memory-efficient, can loop.	DFID combines: DFS memory + BFS optimality guarantee.
Admissible	Monotone	Admissible: $h(n) \leq h^*(n)$. Monotone: $h(n_i) - h(n_j) \leq \text{cost}$. Monotone is stricter.	Monotone $\rightarrow$ Admissible. Admissible does NOT $\rightarrow$ Monotone.
State Space	Search Tree	State Space = abstract/conceptual model of problem. Search Tree = concrete tree built during search.	A search tree is constructed FROM the state space during exploration.
Minimax	Alpha-Beta	Minimax: explores all nodes. Alpha-Beta: prunes useless branches.	Same minimax result; α - β always at least as efficient.
Best-First	A*	Best-First: uses $h(n)$ only — not optimal. A*: uses $f(n)=g(n)+h(n)$ — optimal.	A* is Best-First with the complete evaluation function.
Crossover	Mutation	Crossover: combines two parents. Mutation: randomly changes one individual.	Crossover = main search. Mutation = diversity preserver.

■ EXAM DAY QUICK REFERENCE CARD

INSTANT TRUE/FALSE ANSWERS:

TRUE: α - β ALWAYS $\geq$ Minimax in efficiency

TRUE: TSP is NP-hard

TRUE: Monotone $\rightarrow$ Admissible

TRUE: GA = variant of Hill Climbing

TRUE: DFID finds shortest path

TRUE: A* optimal with admissible h

FALSE: BFS always optimal (only equal costs!)

FALSE: Hill climbing always gives optimal

FALSE: Admissibility $\rightarrow$ Monotonicity

FALSE: Admissibility stricter than monotonicity

FALSE: Larger tournament = more diversity

FALSE: α -cut prunes below MAX node (it's MIN!)

FALSE: Worst-case α - β is $O(b^{d/2})$

MUST MEMORISE FOR EXAM:

- 4-tuple state space: [N, A, S, GD]
- $f(n) = g(n) + h(n)$
- Admissibility: $h(n) \leq h^*(n)$
- Monotone: $h(n_i) - h(n_j) \leq \text{cost}(n_i, n_j)$
- α - β init: $\alpha = -\infty$, $\beta = +\infty$
- 8-puzzle reachable = $9!/2$
- BFS=FIFO, DFS=LIFO, Best-First=Priority Queue
- Monotone $\rightarrow$ Admissible (not reverse)
- GA = Hill Climbing variant
- Mutation ensures DIVERSITY

TOP MISTAKES TO AVOID:

1. α -cut prunes MIN (not MAX) node
2. Admissibility $\neq$ Monotonicity
3. 8-puzzle = $9!/2$ (not 9!)
4. Forget $h(\text{goal})=0$ in monotonicity
5. Uniform cost: $h(n)=0$ NOT $h(n)=g(n)$

■ EXAM STRATEGY

Phase	Strategy	Time
Part 1 MCQs (20 marks)	Answer all 20. Watch traps: Q1, Q7, Q14, Q16, Q17, Q18. Transfer answers to answer sheet.	~20 mins
Q1 State Space (10 marks)	Draw state diagram. List all 4 invalid states. Show full solution path as tuples.	~10 mins
Q2 Heuristics (10 marks)	For (a) use comparison table. For (b) write full summing proof step-by-step.	~10 mins
Q3 Pseudocode (10 marks)	Identify bug (leftmost vs best-heuristic). Explain why monotone removes need for closed list.	~10 mins
Q4 Alpha-Beta (10 marks)	Root: $\alpha = -\infty$, $\beta = +\infty$. Traverse left-to-right. Update α at MAX, β at MIN. Prune when $\alpha \geq \beta$.	~10 mins
Q5 GA/TSP (10 marks)	Show each OX crossover step. Explain tournament size vs diversity tradeoff.	~10 mins

■ COVERAGE: 6 topics · 12 formulas · 20 MCQs · 5 long-answers · all traps highlighted ■ Good luck!